

TRABAJO DE FIN DE GRADO

Orchestration Trade-offs in Edge Computing: Systemd vs Kubernetes

Análisis de rendimiento, distribución de cargas e inferencia de modelos de IA en el extremo de la red

Autor:

[Tu Nombre y Apellidos]

Director/es:

[Nombre del Director/a]

Grado en Ingeniería [Informática / Telecomunicaciones]

[Nombre de tu Universidad]

[Mes y Año de la convocatoria]

Índice

Índice de figuras

Índice de cuadros

Capítulo 1

Introducción

1.1 Contexto y Motivación

El auge del *Edge Computing* plantea nuevos retos en la orquestación de servicios descentralizados. Los entornos perimetrales poseen recursos limitados que cuestionan la viabilidad de modelos *cloud-native* puros.

1.2 Objetivos del Trabajo

El objetivo principal es analizar y comparar los *trade-offs* (rendimiento, consumo y complejidad) entre una orquestación basada en Systemd (nativa y ligera) frente a Kubernetes (robusta pero pesada) en escenarios Edge.

1.3 Estructura de la Memoria

El documento se estructura siguiendo la progresión del desarrollo técnico iterativo, desde la base teórica hasta la comparativa final.

Capítulo 2

Estado del Arte y Tecnologías Involucradas

Este capítulo compila la investigación técnica previa (correspondiente a las *Setmanas 4 y 5*) necesaria para fundamentar el diseño arquitectónico del proyecto.

2.1 Paradigmas de Computación: Cloud vs. Edge Computing

Diferencias entre Cloud, Fog y Edge. Desafíos en el Edge (huella de memoria, latencia, ancho de banda).

2.2 Tecnologías de Contenedorización Ligera

Análisis de Podman frente a Docker, destacando su arquitectura *daemonless* y ejecución *rootless*, ideales para entornos perimetrales seguros y con pocos recursos.

- **Podman:** Arquitectura *daemonless* y *rootless*. Por qué es mejor que Docker para el Edge.

2.3 Orquestación y Ciclo de Vida en el Edge

2.3.1 Systemd y la evolución hacia Quadlets

Gestión del ciclo de vida de los contenedores de forma nativa desde el sistema operativo. Systemd Quadlets integran la gestión del ciclo de vida de los contenedores directamente en el sistema de inicio.

2.3.2 Distribuciones Ligeras de Kubernetes (K3s)

Concepto de K3s como estándar de facto para llevar la semántica de Kubernetes al Edge. Arquitectura del plano de control y el CNI (Container Network Interface).

2.4 Protocolos de Comunicación y Serialización para Alta Frecuencia

2.4.1 Arquitecturas HTTP/REST y RPC (gRPC)

- **REST / HTTP(s) y JSON:** Estándar web, basado en texto. Cuellos de botella en el parseo.
- **gRPC y Protocol Buffers:** Llamadas a procedimientos remotos sobre HTTP/2. Serialización binaria estructurada y fuertemente tipada.

2.4.2 Mensajería Asíncrona y Punto a Punto (ZeroMQ, MQTT)

- **ZeroMQ:** Sockets TCP crudos sin *broker* (Brokerless P2P). Alta velocidad.
- **MQTT:** Patrón Pub/Sub con *broker* intermediario.

2.4.3 Estrategias de Serialización: JSON vs Protobuf vs MessagePack

Comparativa teórica entre texto plano con alto *overhead* y esquemas binarios eficientes.

- **MessagePack:** Serialización binaria sin esquema (*schema-less*), flexible y rápida.

2.5 Frameworks de Machine Learning para Inferencia

Introducción a PyTorch y las características del dataset MNIST utilizado para las cargas de trabajo. Breve introducción a PyTorch, inferencia de modelos, y el dataset MNIST.

Capítulo 3

Metodología y Evolución de la Arquitectura

3.1 Enfoque de Desarrollo Iterativo

El proyecto se ha abordado mediante iteraciones incrementales semanales. Se partió del aprovisionamiento base (*Setmana 0 a 3*), pasando por la optimización de protocolos (*Setmana 4*), la integración de IA (*Setmana 5*) y culminando en la migración a Kubernetes (*Setmana 6*).

3.2 Definición del Entorno de Pruebas

Infraestructura compuesta por un nodo Master (Ubuntu 22.04, WSL) y nodos Workers (Ubuntu 24.04, VMware VMs).

- Nodo Master (Padre): Ubuntu 22.04.5 bajo WSL en Windows 11. Rol: Orquestador (Ansible), entrenamiento de IA, y despachador de tareas.
- Nodos Workers (Hijos): 2 Máquinas Virtuales Ubuntu 24.04.4 en VMware. Rol: Inferencia, cálculo, retorno de métricas.

3.3 Herramientas de Aprovisionamiento y Automatización

Justificación del uso de Ansible para garantizar despliegues reproducibles bajo el patrón de *Side-loading* (evitando repositorios de contenedores externos).

Explica la separación estricta entre *Build* (construcción en el máster) y *Run* (ejecución en el worker). Detalla cómo Ansible transporta los artefactos locales (‘.tar’) usando ‘podman load’ (estrategia *side-loading*) para ahorrar ancho de banda y CPU, evitando repositorios externos (Docker Hub).

3.4 Casos de Uso y Definición de Tareas

3.4.1 Tarea 1: Transmisión masiva de datos (Conteo de imágenes)

División del dataset en N particiones y retorno del conteo.

3.4.2 Tarea 2: Inferencia distribuida de Inteligencia Artificial

Sustitución del conteo simple por la predicción numérica usando modelos pre-entrenados.

3.5 Arquitectura Lógica de la Solución

Aquí debes poner el esquema de la arquitectura Master-Worker, mostrando cómo fluyen los datos (Fase 2, Fase 3, Fase 4).

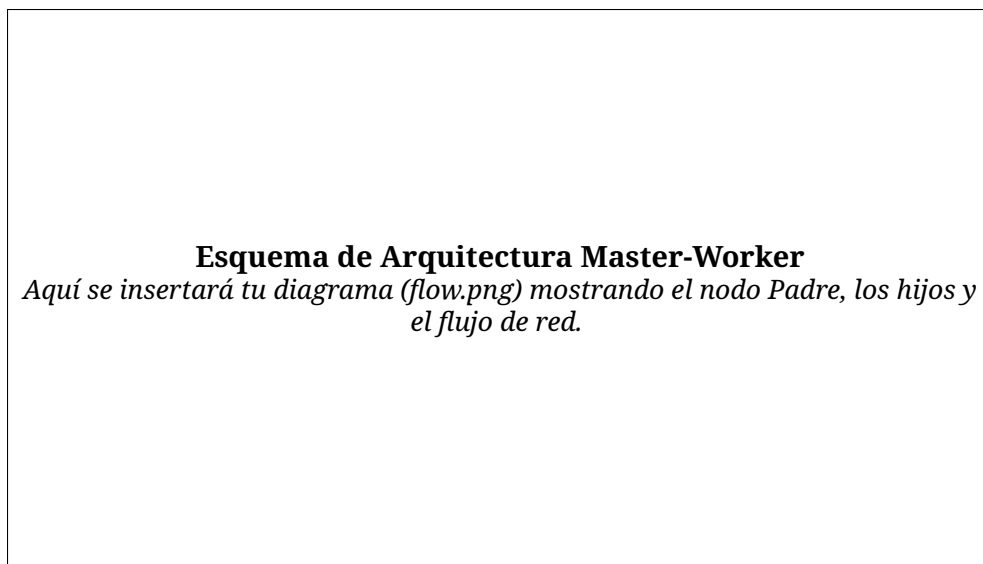


Figura 3.1: Topología del sistema y flujos de comunicación.

Capítulo 4

Fase 1: Arquitectura Base y Optimización de Red (Systemd + Podman)

4.1 Despliegue de la Infraestructura mediante Systemd Quadlets

Explicación de cómo se estructuraron los servicios base de forma agnóstica a la carga de trabajo.

Definición de la Tarea

El Master descarga el dataset MNIST (60,000 imágenes), lo carga en memoria, lo divide en N particiones ($tamaño_particion = total_imagenes / N_nodos$) y envía los lotes. El Worker cuenta las imágenes recibidas y devuelve el total al Master.

4.2 Diseño del Banco de Pruebas de Comunicaciones

Metodología para medir latencias y consumos enviando 60.000 imágenes a los nodos.

- **Línea Base:** HTTP/1.1 + JSON (Demostrar ineficiencia del texto plano).
- **Estándar Microservicios:** gRPC + Protobuf (HTTP/2 binario).
- **Rendimiento Crudo:** ZeroMQ + Protobuf (Límite físico P2P).
- **IoT Broker:** MQTT + Protobuf (Penalización por doble salto).
- **Flexibilidad:** ZeroMQ + MessagePack (Esquema estático vs dinámico).

4.3 Resultados: Comparativa de Rendimiento de Protocolos

Análisis del rendimiento base (HTTP/1.1), el estándar microservicios (gRPC), el crudo P2P (ZeroMQ) y el paradigma IoT (MQTT).

4.4 Resultados: Impacto de la Serialización

La transición de JSON a binario (Protobuf) y el refinamiento final con MessagePack (*schema-less*).

Protocolo	Formato	Tiempo RTT (ms)	Throughput (img/s)	RAM Pico (MB)
HTTP/REST	JSON	≈ 16500	3,636	2600
gRPC	Protobuf	≈ 1037	57,850	384
ZeroMQ	Protobuf	≈ 672	89,285	250
ZeroMQ	MsgPack	≈ 540	111,087	209

Cuadro 4.1: Resumen de resultados de transmisión para 60,000 imágenes.

4.5 Selección de la Arquitectura Óptima para el Edge

Conclusión de esta fase: La elección de ZeroMQ + MessagePack como pila de red ganadora para integrar en Systemd. La tríada ZeroMQ + MessagePack + Zero-Copy se establece como la arquitectura ganadora dentro de Systemd.

Capítulo 5

Fase 2: Implementación del Flujo de Machine Learning

5.1 Entrenamiento y Empaquetado del Modelo (PyTorch)

El nodo Master asume la carga computacional de entrenamiento y serializa el modelo (‘.pth’). Utiliza PyTorch para entrenar un modelo clasificador convolucional (CNN) sencillo sobre el dataset MNIST. Detalla las métricas del entrenamiento (Epochs, Loss, Accuracy final).

5.2 Distribución Segura de Modelos y Datos a Nodos Trabajadores

Envío del modelo y las particiones mediante la arquitectura de red optimizada en la Fase 1. Se envía el modelo entrenado (ej. serializado con ‘torch.save’ en formato ‘.pt’ o ‘.pth’) hacia los workers utilizando ZeroMQ. Seguidamente, se envían las particiones de imágenes.

5.3 Ejecución de Inferencia Descentralizada

Los Workers cargan el modelo en memoria y evalúan cada imagen, devolviendo las predicciones agregadas al Master. Los nodos hijos reciben el modelo, lo cargan en memoria (‘torch.load’), y para cada imagen recibida realizan una inferencia (predicción del número escrito a mano).

5.4 Análisis de Rendimiento y Consumo Computacional

Evaluación del incremento de estrés en CPU y memoria (T_{proc}) al añadir el *overhead* matemático de los tensores.

- Tiempo medio de inferencia por lote.

- Consumo de CPU de los workers durante la carga computacional (Tensores IA).
- Fiabilidad: Exactitud de los números predichos y devueltos al máster.

Capítulo 6

Fase 3: Migración a Kubernetes y Análisis Comparativo Final

6.1 Transición de Arquitectura: De Quadlets a Manifestos de K3s

Adaptación de los servicios a la semántica de *Deployments* y *Services* de K8s.

- Master asume el rol de Control Plane.
- Workers asumen roles de Kubelets.
- Traducción de los servicios ‘Systemd Quadlets’ a manifestos de Kubernetes (‘Deployments’, ‘Services’, ‘Pods’).

6.2 Despliegue del Escenario de Inferencia sobre K3s

Ejecución de la Tarea 2 a través de la interfaz de red de contenedores (CNI) de Kubernetes. Se repite el envío masivo de imágenes y la inferencia de IA, pero ahora a través de la red virtual de Kubernetes (ej. Flannel o Calico CNI) y Kube-proxy.

6.3 Comparativa de Trade-offs: Systemd vs. Kubernetes

6.3.1 Plano de Gestión: Consumo de RAM y CPU

Comparativa del *overhead* en reposo del Kubelet frente al demonio de Systemd. Consumo de RAM y CPU de Systemd+Podman inactivo frente al Kubelet+Kube-proxy+Etcd inactivos en el Edge.

6.3.2 Plano de Datos: Latencia de red y Throughput

Impacto del enrutamiento de Kube-proxy frente a la red nativa punto a punto. Cuánta latencia extra añade el CNI de K8s a gRPC y ZeroMQ frente a la comunicación nativa P2P de Podman.

6.3.3 Complejidad Operativa y Facilidad de Mantenimiento

Facilidad de despliegue usando Ansible estático vs. mantenimiento del estado en K8s.

Capítulo 7

Conclusiones y Trabajo Futuro

7.1 Conclusiones de la Investigación

Resumen de los hitos alcanzados y la viabilidad técnica demostrada a lo largo de los capítulos. Se destacan los números clave: 1) La inviabilidad de REST/JSON. 2) La eficiencia extrema de ZeroMQ+MsgPack en Podman. 3) El *overhead* demostrado por Kubernetes frente al despliegue desnudo.

7.2 Respuesta al Dilema: ¿Systemd o Kubernetes para el Edge?

Veredicto final apoyado en las métricas extraídas en la Fase 3. Cuándo merece la pena asumir el coste de Kubernetes y cuándo es preferible la simplicidad de Systemd.

7.3 Líneas de Trabajo Futuro

Sugerencias para ampliar el trabajo:

- Probar aceleradores hardware en los Workers (GPU/NPU).
- Usar un framework de IA específico para Edge como TensorFlow Lite o ONNX Runtime.
- Implementar mecanismos de tolerancia a fallos automáticos en el lado de Systemd.

Bibliografía

- [1] Red Hat, *Podman Documentation*, [Online]. Disponible en: <https://podman.io/docs/>
- [2] Freedesktop.org, *systemd.service — Service unit configuration*, [Online].
- [3] Cloud Native Computing Foundation (CNCF), *gRPC: A high performance, open source universal RPC framework*, [Online].
- [4] P. Hintjens, *ZeroMQ: Messaging for Many Applications*, O'Reilly Media, 2013.
- [5] Rancher Labs (SUSE), *K3s Lightweight Kubernetes*, [Online]. Disponible en: <https://k3s.io/>
- [6] Paszke, A. et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.